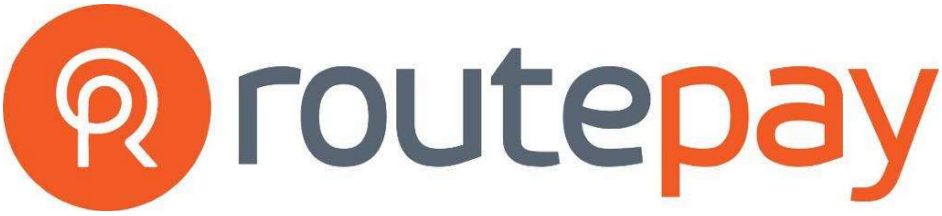
RoutePay API — Developer Guide



API Document v1

Table Of Contents

-
-
- API Document v1
- 1. Overview
- 2. Quickstart — 3 steps
- 3. Environments & Base URLs
- 4. Authentication
- 5. Common headers & conventions
- 6. Payment flow
    - 6.1 Create payment (SetRequest)
    - 6.1 Response sample
    - 6.2 Redirect / Hosted payment page
    - 6.3 Query transaction status
- 7. Bills Payment flow
    - 7.1 Lookup (product info / bundle list)
    - 7.2 Charge (execute payment)
    - 7.2 Response sample
- 8 Payout API
    - 8.1 Create Subaccount
    - 8.2 Get Subaccount Details
    - 8.3 Get Subaccount Balance
    - 8.5 Payouts (Transfer Funds)
    - 8.6 Debits
- 9. Generate Customized QR Dynamic Payment Link
    - Request Format
        - Example curl Request
        - Response
        - Response Fields
        - Field
        - Type

        - Description
- 10. Payment status codes
- 11. Error handling and idempotency
- 12. Testing / Sandbox card data
- 13. Example integrations
- 14. Postman collection & developer tools
- 15. Security & best practices
- 16. Support & contact

This document explains the payment flows, endpoints, authentication and examples for integrating with the RoutePay API.

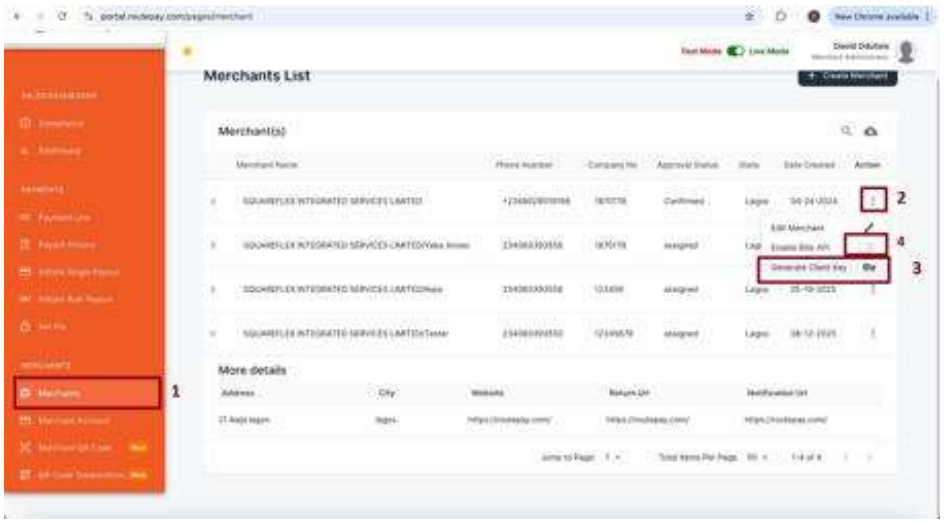
1. Overview

The RoutePay Payment and BillsPayment API enables merchants to initiate, process and verify payments for digital goods and services.

This guide is intended for backend developers integrating merchant systems with RoutePay's APIs. It is based on the official RoutePay Postman collection supplied to developers.

2. Quickstart — 3 steps

1. **Creating a Merchant Application and Generating Credentials in RoutePay Merchant Portal**
    - Log in to the RoutePay Merchant Portal.
    - Navigate to the Merchants section to view the list of all merchants.
    - Select the merchant for which you want to create an application.
    - Click the three-dots menu (⋮) next to the merchant.
    - Choose "Generate Client Secret" from the dropdown options.
    - The system will generate a ClientId and ClientSecret.
    - Copy and store these credentials securely — they are required for API integration.



2. Request an access token (OAuth2 client_credentials) from the auth server and include Authorization: Bearer <access_token> on all API calls.
3. Use SetRequest to create a payment or bills/api/v1/Payment/charge for airtime/bundle purchases, and always confirm final payment status with the GetTransaction endpoint.

Note: Ensure that the "**Enable Bills API**" option is toggled ON in step 4.

3. Environments & Base URLs

Sandbox: https://apiqa.routepay.com

Production: https://api.routepay.com

(Where relevant services use subdomains such as auth{Env}.routepay.com for token issuance and payment{Env}.routepay.com for hosted payment pages in dev/test.)

---

4. Authentication

Token endpoint: POST https://auth{{Env}}.routepay.com/connect/token (application/x-www-form-urlencoded)

Request fields:

- grant_type=client_credentials
- client_id
- client_secret

Sample success response includes

- access_token
- token_type
- expires_in seconds.

  Store securely and use as a Bearer token.

5. Common headers & conventions

Use Content-Type: application/json for JSON payloads.

Include Authorization: Bearer {{access_token}} for protected endpoints.

Use merchantReference and transactionReference in your system to correlate requests and responses.

Confirm successful payment using both HTTP 200 and payment-specific status values (see section 9).

---

6. Payment flow

6.1 Create payment (SetRequest)

Endpoint: POST {{baseUrl}}/payment/api/v1/Payment/SetRequest

POST {{baseUrl}}/payment/api/v1/Payment/SetRequest

Content-Type: application/json

Authorization: Bearer {access_token}

Request body example

```
{
  "merchantId": "{{ClientId}}",
  "returnUrl": "https://merchant.example.com/return",
  "merchantReference": "A1B2",
  "totalAmount": "500",
  "currency": "NGN",
  "paymentType": "PAYMENT",
  "customer": {
    "email": "test@example.com",
    "mobile": "0800000000",
```

```
POST /api/v1/Subaccounts/{accountNumber}/debits
```

Path Parameter:

```
accountNumber (string, required) — Subaccount Number.
```

Request Body:

```
{
  "transferNarration": "string",
  "amount": 0,
  "transferReference": "string"
}
```

Response (200 – Success):

```
{
  "isSuccess": true,
  "message": "Debit successful",
  "responseCode": "00"
}
```

9. Generate Customized QR Dynamic Payment Link

This API generates a **customized QR code** for a payment request to a merchant. The QR code can be scanned by customers to initiate and complete payment.

Endpoint

```
Post:
{baseurl}/api/v1/QRCode/GenerateCustomizedQRDynamicPaymentLink
Authentication
Type: Bearer Token (JWT)
Header:
Authorization: Bearer <your_access_token>
```

Request Format

The API accepts a multipart/form-data payload.

Parameters

Field	Type	Required	Description
DynamicName	string	Yes	A dynamic identifier for the QR code (e.g., customer name, label).

Amount	decimal	Yes	The payment amount to be linked to the QR code.
DurationInSeconds	number	Yes	Expiry duration of the QR code in seconds (e.g., 259200 = 3 days).
MerchantReference	string	Yes	Merchant's unique reference for tracking the transaction.
MerchantLogo	file	Yes	Merchant's logo image (must be PNG format).
ReturnUrl	string	Yes	The callback URL to redirect after payment is completed.
VerifyMe	boolean	No	Flag to enable extra verification on the payer (default: true).

Example curl Request

```
curl --location
'{baseurl}/api/v1/QRCode/GenerateCustomizedQRDynamicPaymentLink' \
--header 'Authorization: Bearer <your_access_token>' \
--form 'DynamicName="Rli"' \
--form 'Amount="2500"' \
--form 'DurationInSeconds="259200"' \
--form 'MerchantReference="INV-20251001-1246"' \
--form 'MerchantLogo=@"///Mac/Home/Downloads/download.png"' \
--form 'ReturnUrl="https://routepay.com "' \
--form 'VerifyMe="true"'
```

Response

```
Success Response
{
  "value": {
    "qrCode":
"https://staging.qr.routepay.com/QrCodes/images/175940007568de508b0c114.png",
    "transactionReference": "2025100211141581551"
  },
  "isSuccess": true,
  "error": "",
  "message": null,
  "responseCode": null
}
```

Response Fields

Field	Type	Description
value.qrCode	string	URL link to the generated QR code image (PNG).
value.transactionReference	string	Unique reference ID for the generated payment transaction.

isSuccess	bool	Indicates if the request was processed successfully.
error	string	Error message if the request fails.
message	string	Optional human-readable message.
responseCode	string	Optional response code for debugging or tracking.

Error Responses

Code	Description
400	Bad request (missing/invalid parameters).
401	Unauthorized (invalid or missing bearer token).
415	Unsupported media type (only PNG is allowed).
500	Internal server error.

Notes

MerchantLogo must be PNG format, otherwise the request will fail with a 415 Unsupported Media Type.

The generated QR code is time-limited based on the DurationInSeconds parameter.

Always store and track the transactionReference for reconciliation.

10. Payment status codes

Always query the transaction — a payment is considered successful only when HTTP 200 AND PaymentStatus = 0 AND Description = "Successful".

Status Code	Meaning
0	Successful
550	Failed
250	Pending
260	Processing
210	AlreadyProcessed
220	Cancelled